Parikshit: agents and Fetch.ai work doc

Oct 24, 2026 · @Chaudhari kevin

Mission and setup

You make Rally's agents real Fetch.ai agents: registered on Agentverse, talking over the network, and usable from ASI:One chat. This is the Fetch.ai track, the one Rally fits best, so your P0 tasks decide whether Rally is eligible for it.

- Branch: `feat/agents-network`, created from `main` after Kevin merges the contracts PR (target 3:30 pm).
- You own: `agents/` and `services/api/rally/negotiation/`. Add your service methods in a marked block at the end of `services/api/rally/service.py`.
- Don't edit: `apps/web/`, `convex/`, `compiler.py`, `matching.py`. Ask Kevin for any UI change; contract changes go through Kevin.

Setup:

```
git checkout main && git pull && git checkout -b feat/agents-network
python scripts/rally.py setup
pip install uagents uagents-core httpx
python scripts/rally.py test          # 131 pass before you start
python agents/bureau_demo.py           # real uAgents, local: agreed on v4 in
4 rounds
```

Accounts: an Agentverse account (agentverse.ai) for mailboxes and agent pages, and ASI:One (asi1.ai) to test the chat. Read the Chat Protocol and mailbox pages on Fetch.ai's docs before task 1; check the exact import paths for your installed `uagents` version, since they move between releases.

What exists already: `negotiation/mediator.py` (bounded single-text mediator), `negotiation/personal.py` (personal agent decisions), `negotiation/transport.py` (in-process transport), `agents/protocol.py` (uAgents wire models), `agents/bureau_demo.py` (a working `UAgentsTransport`). Agent messages are logged with `Rally.agent_msg(...)` in `service.py`, and the UI shows them automatically.

Tasks, in this order

#	Task	Priority	Hours	Due
1	Agents on the network (uAgents transport + Agentverse)	P0	5	9 pm, merge 1
2	Rally concierge on ASI:One (Chat Protocol)	P0	5	2 am, merge 2
3	Group-size nudge from vendor agents	P1	3	2 am
4	Auto-replace a member who leaves	P1	2	2 am
5	Payment Protocol booking fee on ASI:One	P2	3	only if 1 to 4 are merged

1. Agents on the network

Goal: the API stops calling personal agents as Python objects and sends real uAgents messages instead, with every agent visible on Agentverse.

1. Move `UAgentsTransport` from `agents/bureau_demo.py` into `services/api/rally/negotiation/transport.py`. Keep the same interface as `InProcTransport`: `set_plan_space(...)` and `gather(...)`.
2. Create `agents/run_agents.py`: one process that starts a personal agent per seeded intent and one provider agent per vendor, each with a fixed seed from `AGENT_SEED_PREFIX`, `mailbox=True`, and its private prefs loaded from `data/fixtures/private_prefs.json`. Personal agents reuse `PersonalAgentBrain`; provider agents answer `QuoteRequestMsg` from `providers.json`.
3. Start the mediator agent inside the API process (FastAPI lifespan, running alongside uvicorn's event loop). It is the only agent that writes through `service.py`.
4. Choose by flag: `TRANSPORT=inproc` (default, unchanged) or `TRANSPORT=agentverse`. Set `provenance()["transport"]` to the real mode so the UI badge reads "Agents: agentverse".
5. Register every agent on Agentverse with a name like "Rally · Ana's agent", a short description, and Innovation Lab tags.
6. Write each agent's real address into `Member.agent_address` and the diagnostics list, so the agent conversation shows real addresses.

Test: `tests/integration/test_transport_agentverse.py`, skipped unless `TRANSPORT=agentverse`, runs the scuba deal over the network. It must reach plan v4 with the same 4 rounds as in-process. Record per-hop latency into `AgentMessage.latency_ms`.

Done when the scuba demo runs end to end with `TRANSPORT=agentverse` on the venue Wi-Fi, and every agent has an Agentverse page.

2. Rally concierge on ASI:One

Goal: a judge types a goal in ASI:One and gets back a negotiated group plan with a link to the group room. Fetch.ai's rules say the use case must work through ASI:One, with no custom frontend required.

1. New `agents/concierge.py`: an agent with the Chat Protocol (`ChatMessage` / `ChatAcknowledgement`, published manifest) and a mailbox, so ASI:One can discover it.
2. On a message: acknowledge; call the Rally API over HTTP (`/v1/goals:compile`, then `PATCH` the goal if a question is missing and the user's text answers it, `/v1/intents`, `/v1/intents/{id}/matches`, `/v1/deals`, `/v1/deals/{id}/negotiate`); poll `/v1/deals/{id}/snapshot` until `AWAITING_APPROVAL` or `NO_AGREEMENT`.
3. Reply in chat in three short messages: what Rally understood, who matched and who didn't (with reasons), then the agreed plan with price per person and a link: `https://<public web URL>/deals/<id>`.
4. If the compiler asks a question (for example group size), ask it in chat and continue on the user's next message. Keep per-sender state keyed by the ASI:One sender address.
5. Identity: map each ASI:One sender to a Rally persona. For the demo, map the first sender to Kevin and say so in the README.
6. Give the agent a clear Agentverse description and keywords ("group trips", "scuba", "skydiving", "kayaking", "find people", "group discount") so ASI:One routes to it.

Test: a unit test with a fake API client covering the three replies and the follow-up question. Then a manual ASI:One session; save its URL for Devpost.

Done when an ASI:One chat produces a scuba plan and the link opens the live group room.

3. Group-size nudge

Goal: when one more person would move the group into a cheaper price tier, the vendor's agent says so and the room offers to invite the next match.

1. After the binding quote, the provider agent checks `ProviderSheet.tier_for(n + 1)`. If the per-person price drops and `n + 1 ⩽ groupMax`, it sends a `NudgeMsg` (add it to `agents/protocol.py` and `packages/contracts/rally_contracts/messages.py` through Kevin).
2. The mediator stores it in `Deal.nudge` (`providerId`, `extraPeople`, `newPricePerPerson`, `savingPerPerson`) and logs it with `agent_msg`.
3. `POST /v1/deals/{id}/nudge/accept` (organizer only) invites the next eligible person from the stored matches and reruns negotiation; approvals reset as they do today.

Test: a unit test with a 4-person group under tiers where 5 is cheaper; checks the nudge values and that the savings maths is right.

4. Auto-replace a member

When someone leaves during negotiation or approval and a group-size tier or `groupMin` is at risk, the mediator invites the next eligible person from the stored matches, appends to `Deal.replacements`, and renegotiates. Today `leave()` in `service.py` only renegotiates with fewer people.

Test: extend `test_leaving_voids_approvals_and_renegotiates`; the group size stays 5 and the new member's name appears in `replacements`.

5. Payment Protocol booking fee (P2)

Only after 1 to 4 are merged. Use Fetch.ai's Payment Protocol on the concierge to charge a small, clearly labelled testnet booking fee before it starts a negotiation. This answers the "well-defined monetization approach" judging question. Check the current Payment Protocol docs first; if it needs more than 3 hours, write the monetization approach in the README instead.

Handoffs, fallbacks and checklist

What	To	When	How
`NudgeMsg` and the nudge accept endpoint	Kevin (UI card in the group room)	By midnight	Post the JSON shape in the team chat once the endpoint works

What	To	When	How
Real agent addresses and Agentverse URLs	Kevin (README, Devpost)	9 pm	A table in `agents/README.md`: name, address, Agentverse link
Concierge's public web URL for links	From Kevin	9 pm	`RALLY_WEB_URL` env var
"Why this plan" facts from the personal agent	Dev	9 pm	Add `PersonalAgentBrain.explain(plan)` returning per-term fit (term, value, good / ok / compromise), no weights. Dev turns it into words

Fallbacks:

- Agentverse unreachable from the venue network: keep `TRANSPORT=inproc` for the live demo, show the Agentverse pages and the recorded ASI:One session in the video, and say so on stage.
- ASI:One doesn't route to the concierge in time: demo it through the concierge's Agentverse chat page, and keep the session URL as proof.
- Mediator is slow over the network: raise the per-member timeout from 20 s, and keep `DEMO_ROUND_DELAY_MS` low so the total stays under 60 s.

Final checklist:

- [ ] `python scripts/rally.py test` passes on `feat/agents-network`
- [ ] Scuba demo reaches v4 with `TRANSPORT=agentverse`
- [ ] Every agent registered on Agentverse; addresses in `agents/README.md`
- [ ] README has both badges: `![tag:innovationlab](https://img.shields.io/badge/innovationlab-3D8BD3)` and `![tag:hackathon](https://img.shields.io/badge/hackathon-5F43F1)`
- [ ] ASI:One chat session URL saved for Devpost
- [ ] Nudge and auto-replace merged with tests
- [ ] `python tests/evaluation/run_negotiation.py 50` rerun; numbers committed, 0 violations

Rules for the Fetch.ai track are from their LA Hacks 2026 hackpack; Kevin confirms the Cal Hacks 13.0 version at the booth.